Transactional Outbox

Outbox · Atomically save order + outbox row, relay later · Build with Waqar

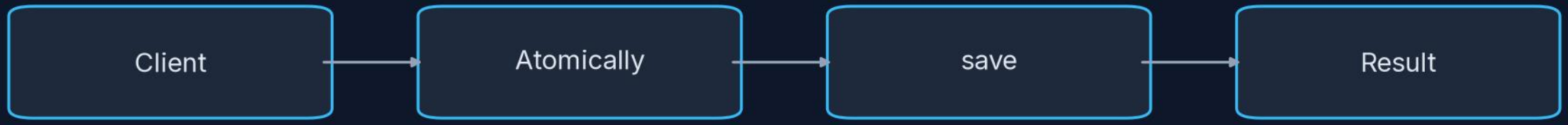
Today's Hook

- Problem: Atomically save order + outbox row, relay later
- Pattern: Outbox — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Atomically save order + outbox row, relay later
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Outbox: Atomically save order + outbox row, relay later

Real-World Scenario

- Running example: Atomically save order + outbox row, relay later
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.enterprise.outbox.problem;

import java.util.*;

public final class NaiveOrderPub {
  public static void place(String id){ /* save order */ System.out.println("saved "+id); /* publish may fail after */ throw new RuntimeException("broker down"); }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.enterprise.outbox.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
import java.util.function.*;

record OutboxRow(String id, String payload, boolean published){}
final class OutboxStore {
  private final List<OutboxRow> rows=new ArrayList<>();
  private final List<String> orders=new ArrayList<>();
  void placeOrder(String id, String payload){ orders.add(id); rows.add(new OutboxRow(id, payload, false)); }
  List<OutboxRow> unpublished(){ return rows.stream().filter(r->!r.published()).toList(); }
  void markPublished(String id){
    for (int i=0;i<rows.size();i++) if(rows.get(i).id().equals(id)) rows.set(i, new OutboxRow(id, rows.get(i).payload(), true));
  }
  List<String> orders(){ return List.copyOf(orders); }
}
final class OutboxRelay {
  private final OutboxStore store; private final List<String> published=new ArrayList<>();
  OutboxRelay(OutboxStore store){ this.store=store; }
  int relayOnce(){ int n=0; for(var r: store.unpublished()){ published.add(r.payload()); store.markPublished(r.id()); n++; } return n; }
  List<String> published(){ return List.copyOf(published); }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.enterprise.outbox.problem;
import java.util.*; import java.util.concurrent.*;
public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== Outbox: order + event atomically ===");
    System.out.println("--- naive ---");
try { NaiveOrderPub.place("o1"); } catch(Exception e){ System.out.println("caught="+e.getMessage()); }
System.out.println("--- outbox ---");
var store=new OutboxStore(); store.placeOrder("o1","OrderPlaced:o1");
var relay=new OutboxRelay(store); System.out.println("relayed="+relay.relayOnce()); System.out.println("published="+relay.published());
System.out.println("relayedAgain="+relay.relayOnce());
  }
}
```

Walkthrough Output

```
=== Outbox: order + event atomically ===
--- naive ---
saved o1
caught=broker down
--- outbox ---
relayed=1
published=[OrderPlaced:o1]
relayedAgain=0
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Outbox.

Related Patterns

- Contrast Outbox with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Outbox → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.